Web Application Security Assessment Report

Prepared By: Mehedi
Target Application: Damn Vulnerable Web Application (DVWA)
Environment: Localhost (XAMPP on Windows)
Date: May 16, 2026

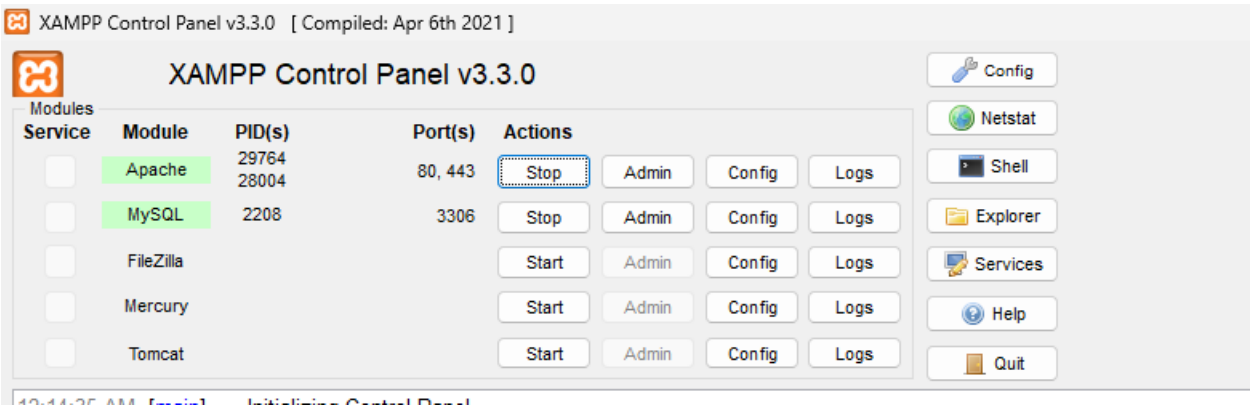
1. Executive Summary

This report documents the setup of a local security laboratory and the subsequent vulnerability assessment of the hosted web application. The environment was built using XAMPP on Windows to host the Damn Vulnerable Web Application (DVWA). The primary objective was to demonstrate and document critical security flaws, specifically focusing on SQL Injection.

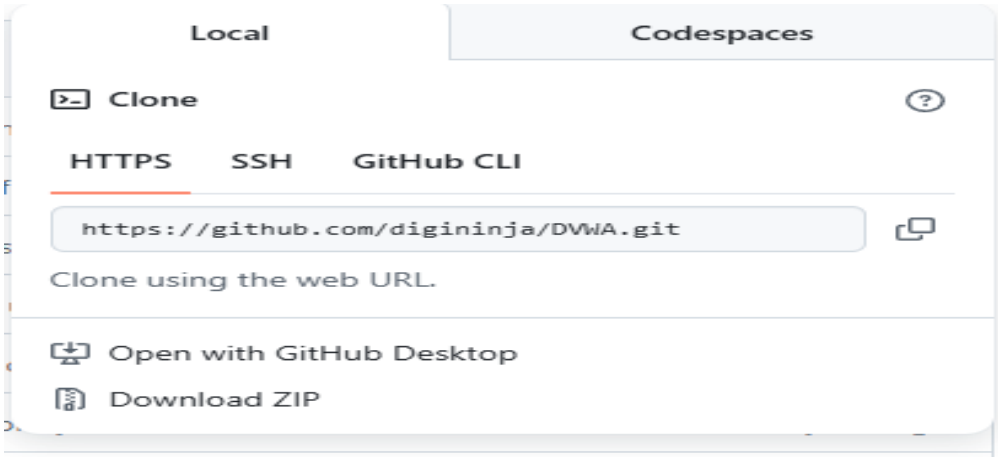
2. Project Phases (Step-by-Step)

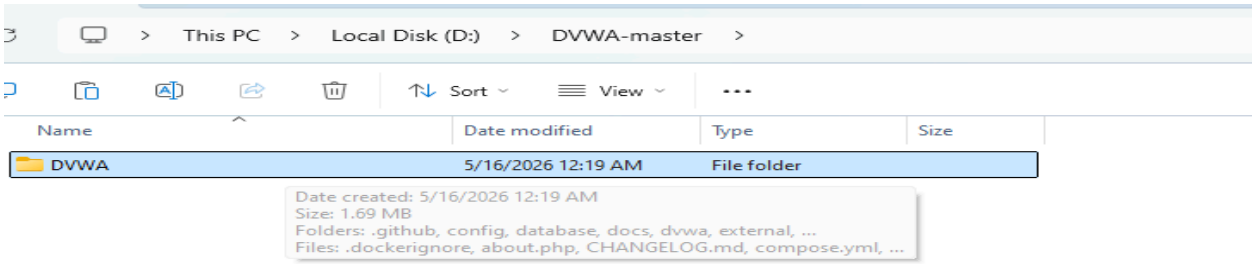
Phase 1: Lab Environment Setup

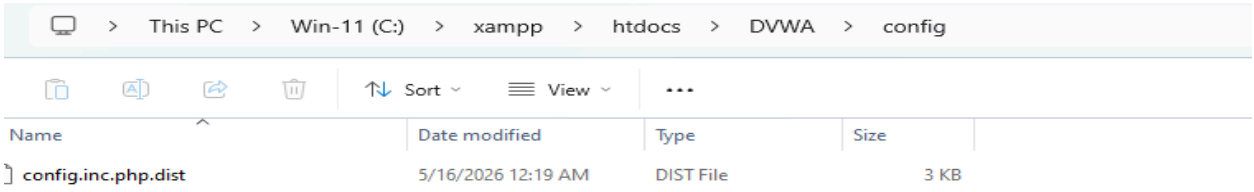
● XAMPP Installation: Installed XAMPP to manage Apache (Web Server) and MySQL (Database).



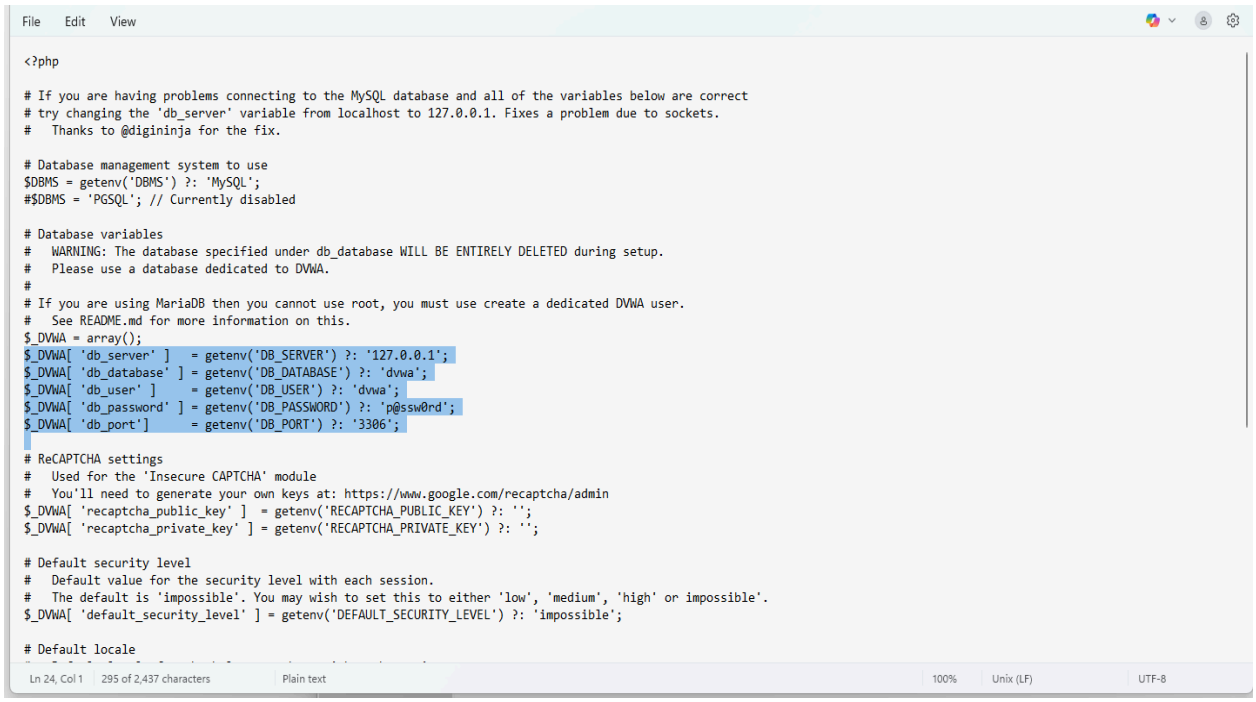
● DVWA Deployment: Cloned the DVWA repository from GitHub and moved the files to the htdocs directory.







● Configuration: Renamed config.inc.php.dist to config.inc.php and configured database credentials (user: 'dvwa', password: 'p@ssw0rd').



● PHP Tuning: Modified php.ini to set allow_url_include to 'On' for application compatibility.

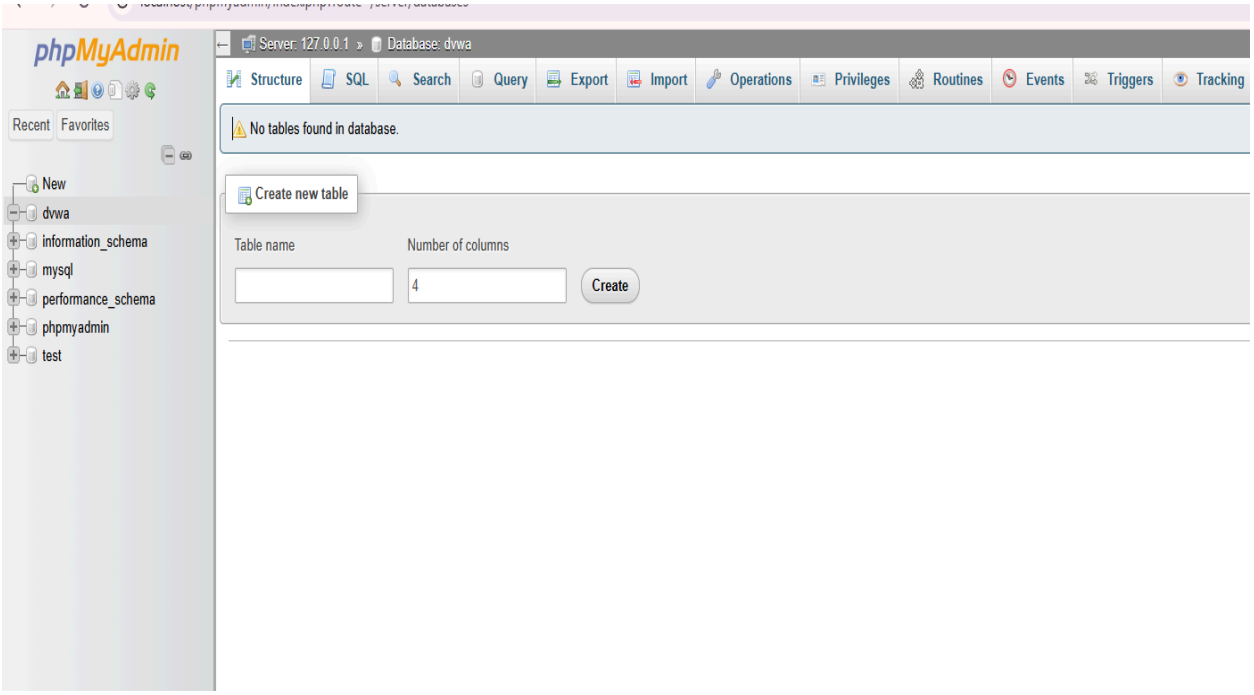
```
; Whether to allow the treatment of URLs (like http:// or ftp://) as files.
; https://php.net/allow-url-fopen
allow_url_fopen=On

; Whether to allow include/require to open URLs (like https:// or ftp://) as files.
; https://php.net/allow-url-include
allow_url_include=On
```

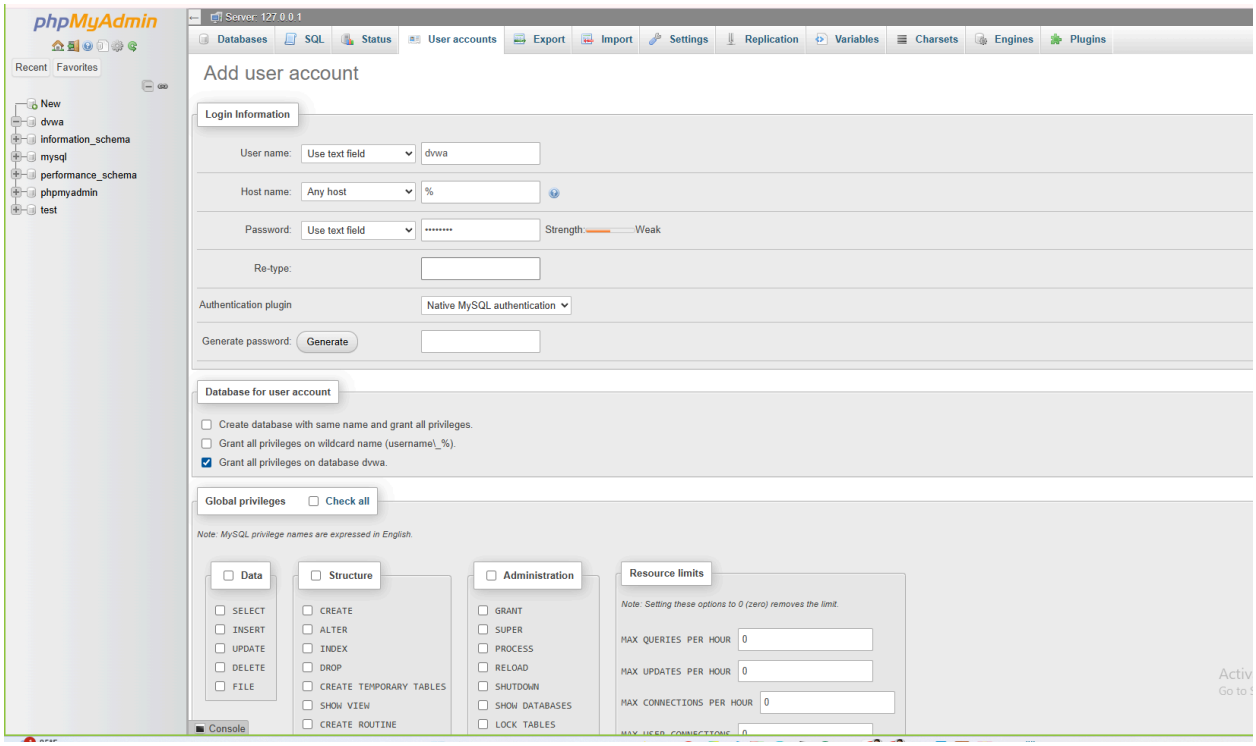
allow_url_include

PHP
PHP version: 8.2.12
PHP function display_errors: Enabled
PHP function display_startup_errors: Enabled
PHP function allow_url_include: Enabled - Feature deprecated
PHP function allow_url_fopen: Enabled
PHP module gd: Installed
PHP module mysql: Installed
PHP module pdo_mysql: Installed

Phase 2: Database Initialization & Troubleshooting

● User Privileges: Created a dedicated database user 'dvwa' in phpMyAdmin and granted all privileges to resolve initial connection errors.





● Database Creation: Successfully initialized the database tables through the DVWA setup page.

Phase 3: Security Configuration
Accessed the 'DVWA Security' panel and set the security level to 'Low' to facilitate the initial vulnerability testing phase.

3. Security Finding: SQL Injection (Critical)

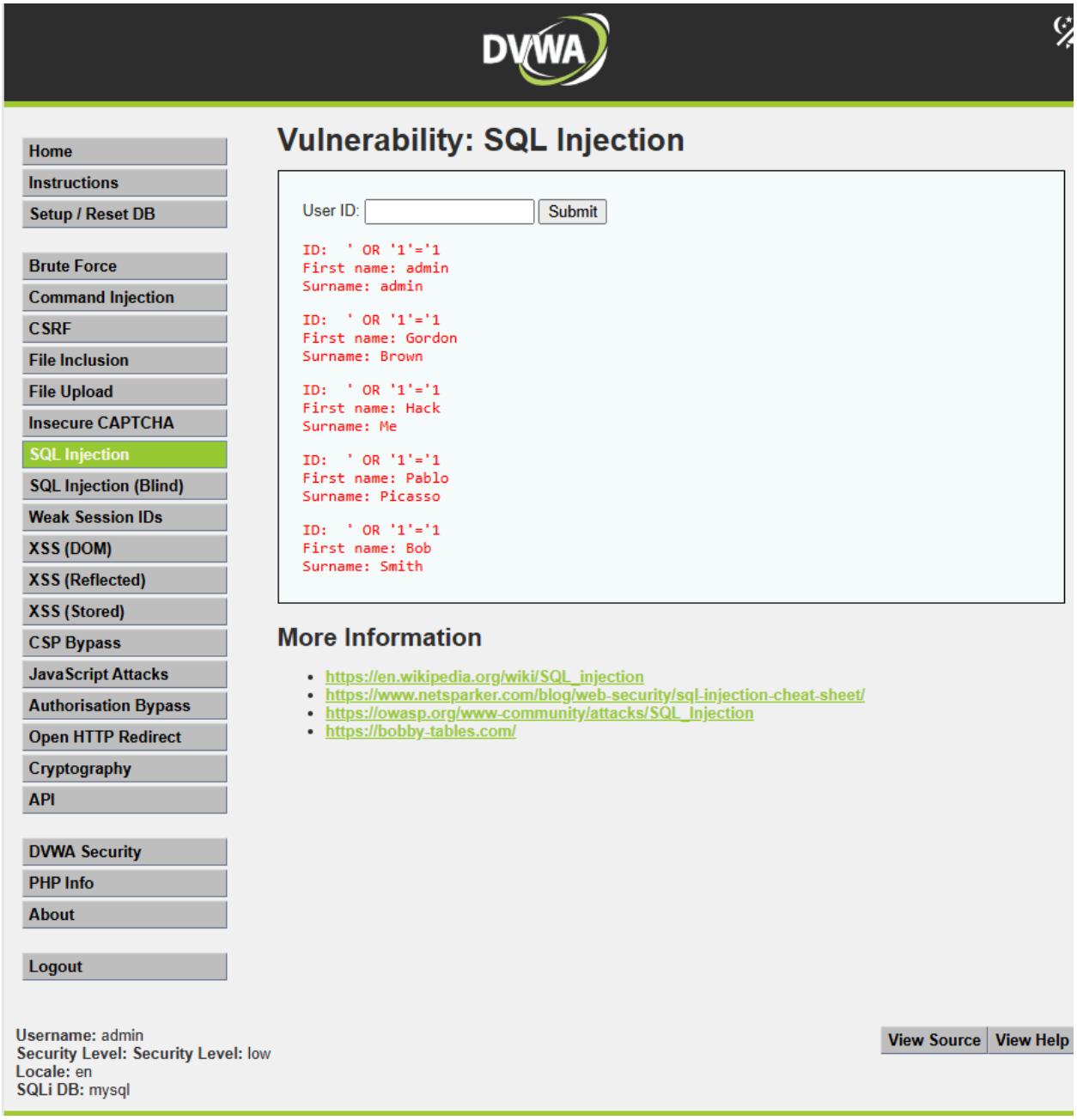
Vulnerability Name	SQL Injection (In-band/Error-based)
Severity	CRITICAL
Vulnerable Component	User ID Input Field

Description:
The application fails to properly sanitize or filter user-supplied input in the 'User ID' field. This allows an attacker to inject malicious SQL commands that alter the intended query logic on the backend database.

Proof of Concept (PoC):

Payload Used: ' OR '1'='1

By entering this payload into the User ID field, the application returned the full list of users registered in the database, bypassing the intended single-user lookup logic.



4. Impact

- Unauthorized Data Access: Full disclosure of sensitive user information.
- Potential Account Takeover: Access to password hashes for offline cracking.

5. Recommendation

Implement Parameterized Queries (Prepared Statements) to ensure that user input is never interpreted as executable code. Additionally, enforce strict input validation and least-privilege database access.